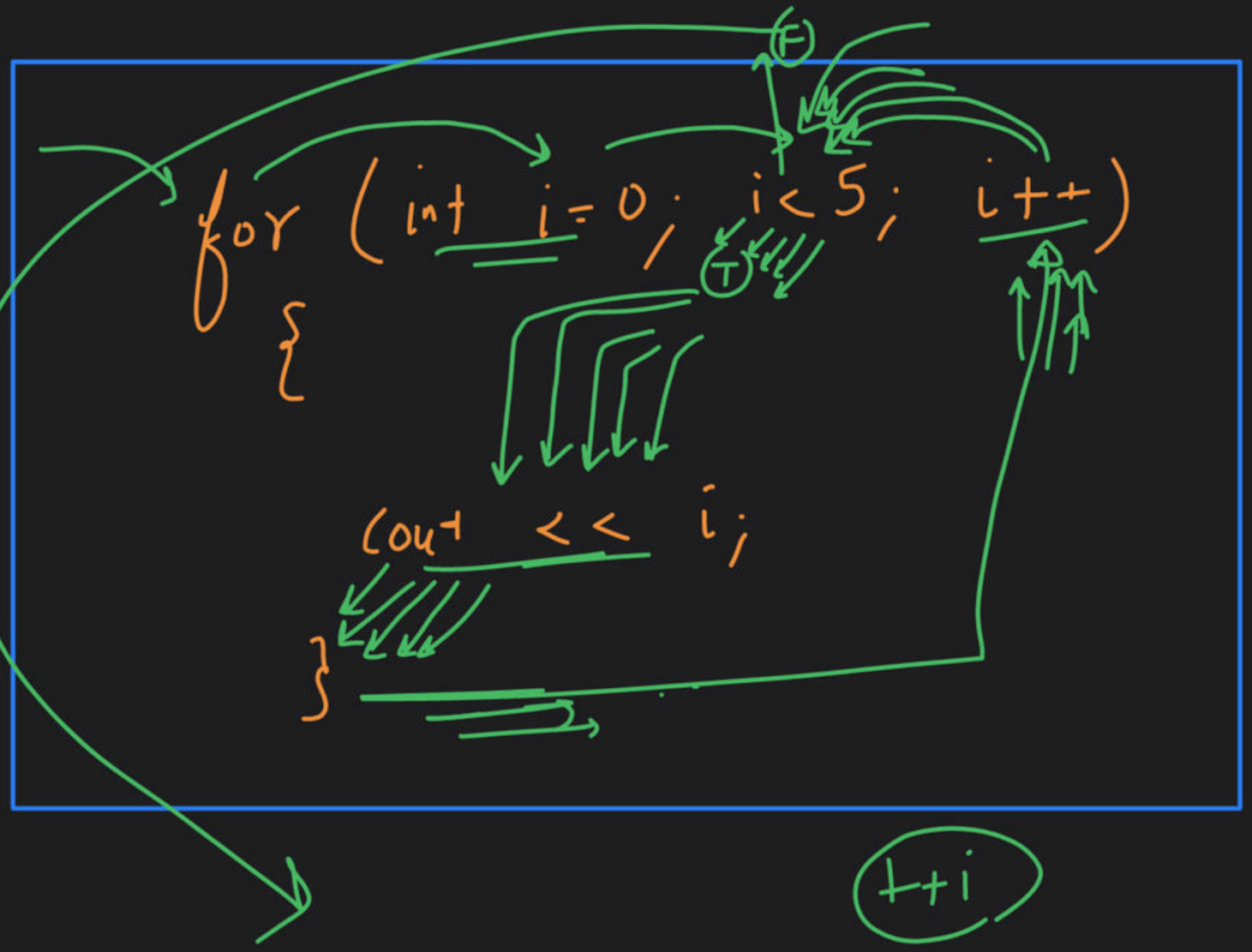


Patterns :->

loop



i = 0

0 < 5 → T

print 0

i = 1

1 < 5 → T

print 1

i = 2

2 < 5 → T

print 2

i = 3

3 < 5 → T

print 3

i = 4

4 < 5 → T

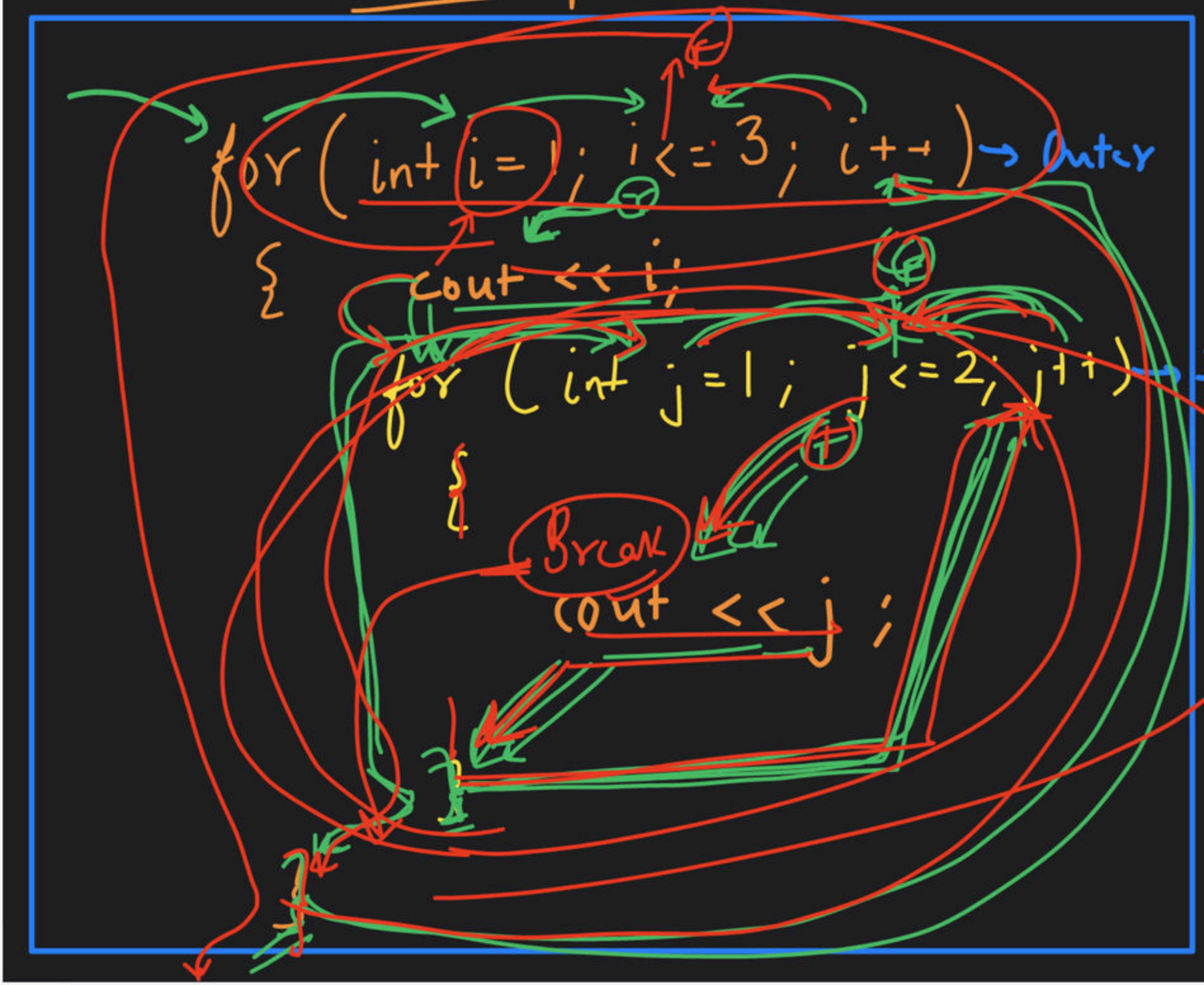
print 4

i = 5

5 < 5 → F

→ Nested Loop :-

i = 4
4 <= 3 → F

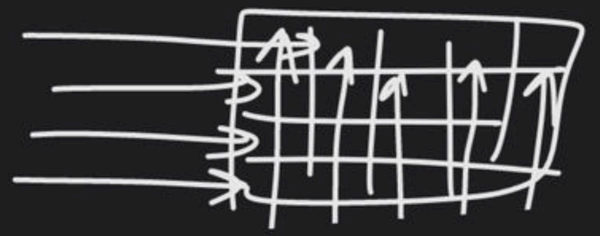


i = 1
1 <= 3 → T
print i = 1
J = 1
1 <= 2 → T
print j = 1
J = 2
2 <= 2 → T
print J = 2
J = 3
3 <= 2 → F

i = 2
2 <= 3 → T
print i = 2
J = 1
1 <= 2 → T
print J = 1
J = 2
2 <= 2 → T
print J = 2
J = 3
3 <= 2
F

i = 3
3 <= 3 → T
print i = 3
J = 1
1 <= 2 → T
print J = 1
J = 2
2 <= 2 → T
print J = 2
J = 3
3 <= 2 → F

Patterns :-



① Square Pattern

Rule

① divide row/columns

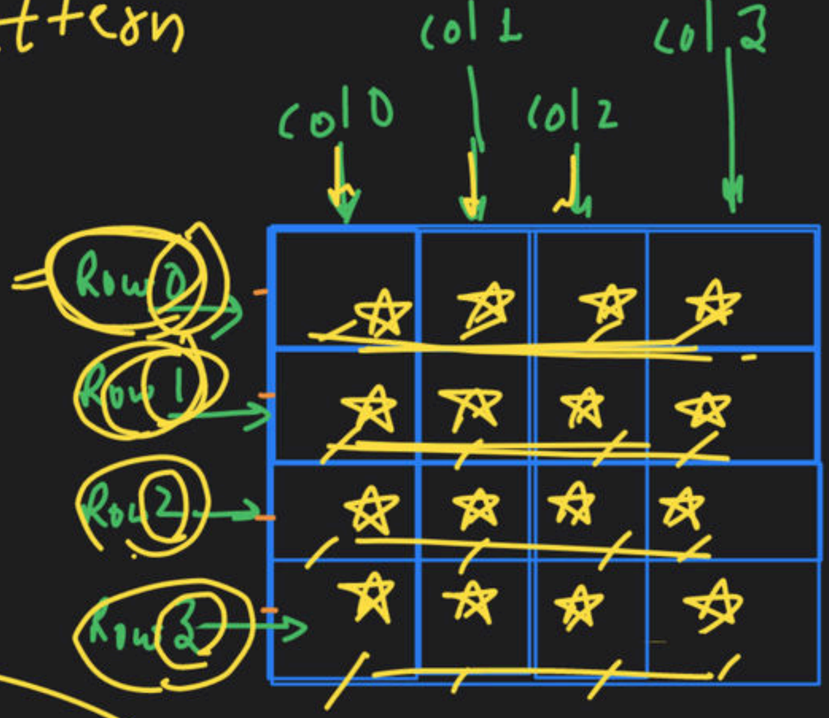
② no of rows → 4 rows

row → Outer loop
col → Inner loop

③

row 0 → 4 ☆
row 1 → 4 ☆
row 2 → 4 ☆
row 3 → 4 ☆

every row
4 ☆ print



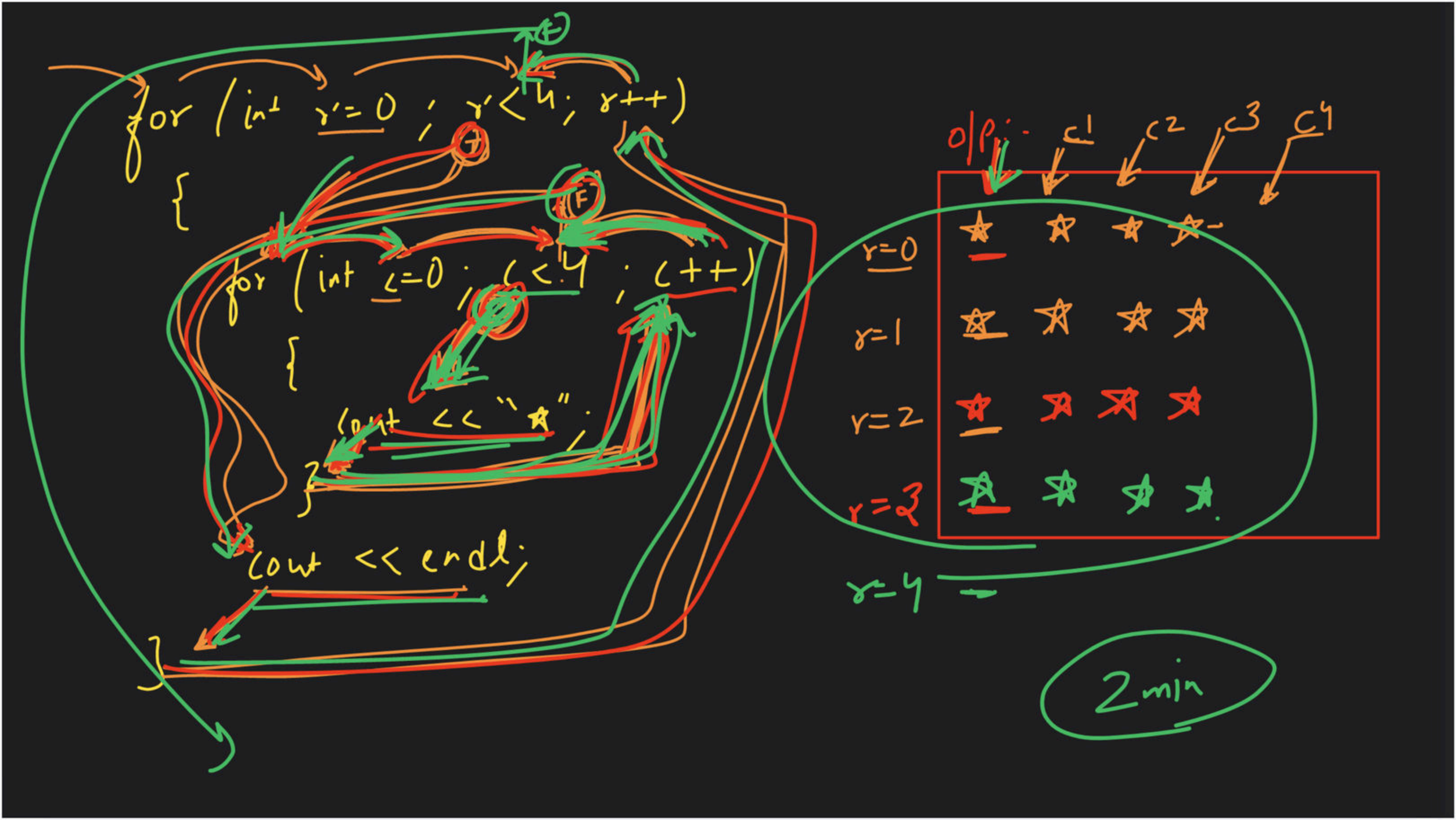
i = 1
j = 1 to 2

i = 2
j = 1 to 3

i = 3
j = 1 to 4

row = 0
j = 0 to [1, 2, 3]

row = 1

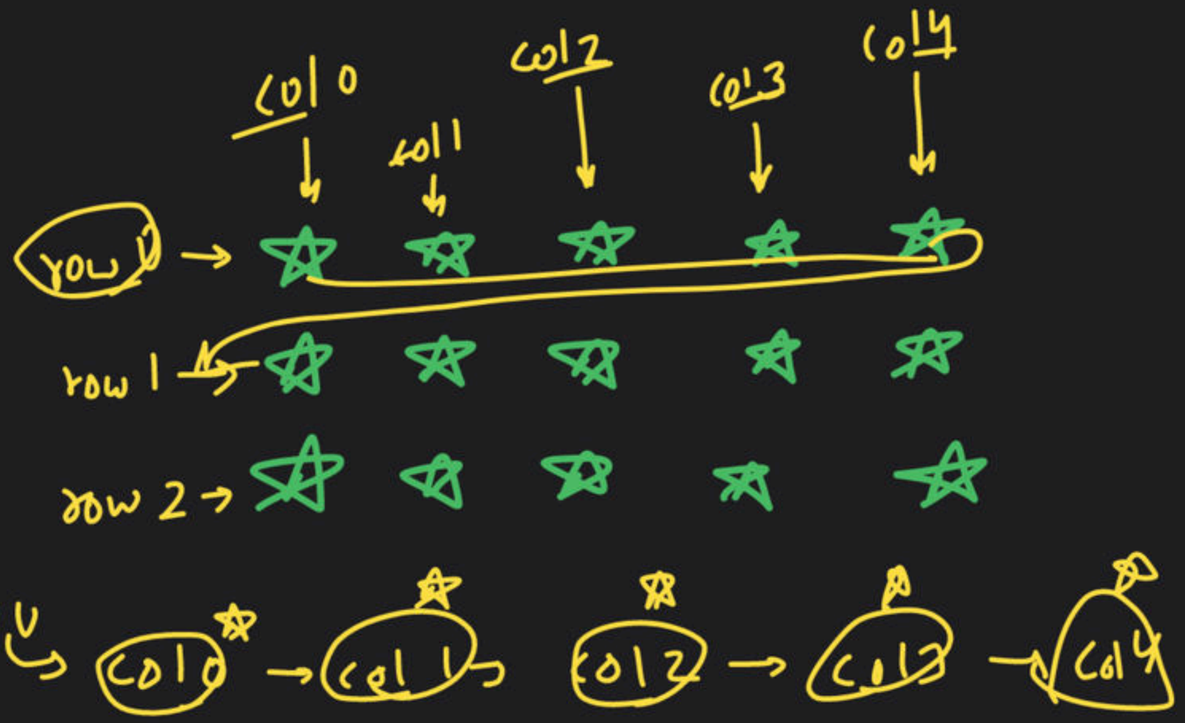

for (int r=0 ; r<4 ; r++)
{
for (int c=0 ; c<4 ; c++)
{
cout << "*";
}
cout << endl;
}
T
F
o/p :
c1
c2
c3
c4
r=0
r=1
r=2
r=3
r=4
2 min

Rectangle

① no of row → 3
↳ outer

② r0 → 5 ★
r1 → 5 ★
r2 → 5 ★

each row
↳ 5 ★




```
for (r=0 → r<3)
{
    for (c=0 → c<5)
    {
        cout << "*";
    }
    cout << endl;
}
```

Hollow Rectangle

→ span

→ total rows → n=4

→

row 0 → 5 ☆

row 1 → 1☆ 3sp 1☆

row 2 → 1☆ 3sp 1☆

row 3 → 5☆

① first & Last row → 5☆

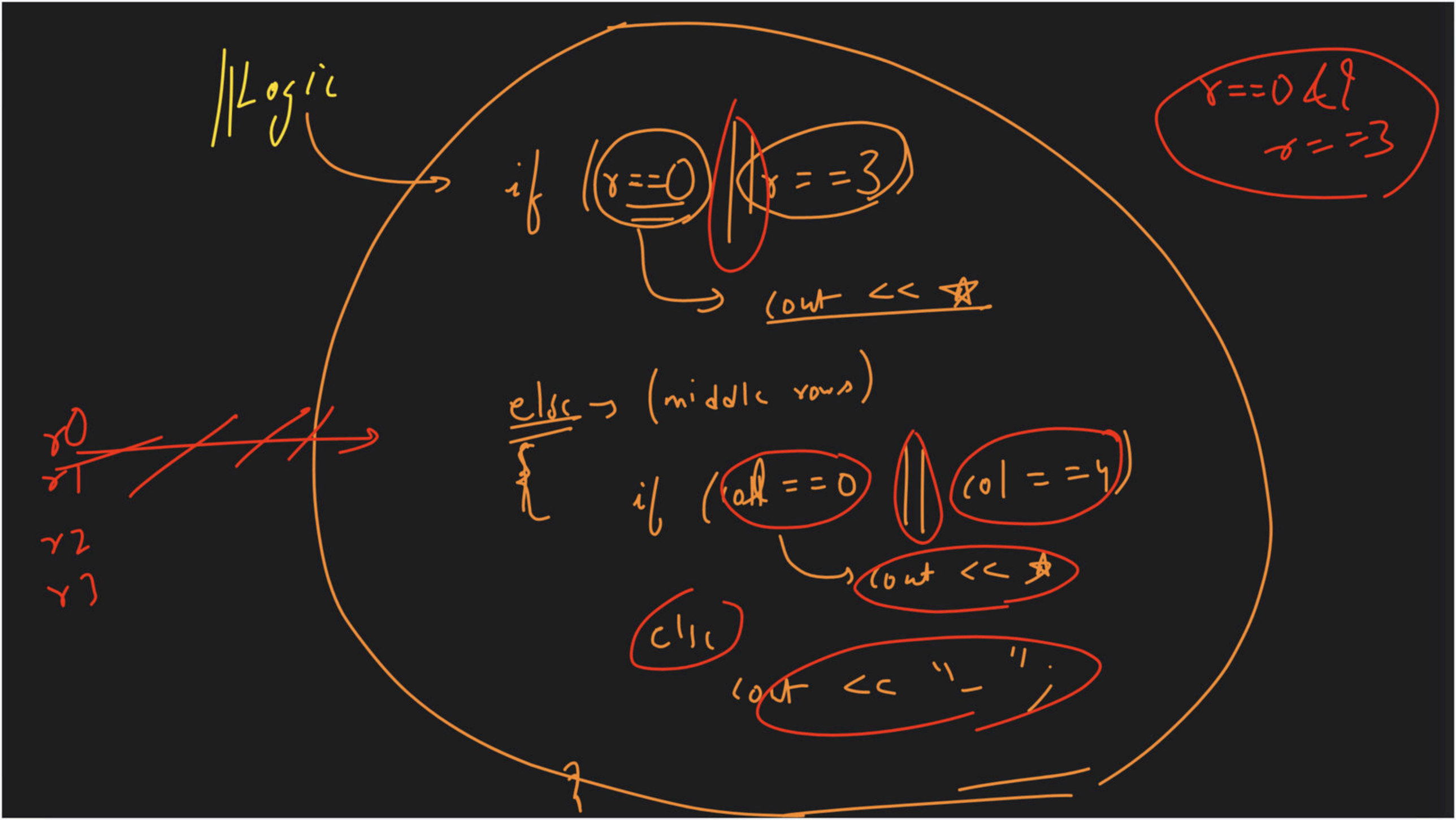
next rows

first & last col → star

etc → space




```
for (int r=0; r<4; r++)
{
    for (int c=0; c<5; c++)
    {
        // Logic
    }
}
```


//Logic
if (r==0 || r==3)
cout << *
r==0 && r==3
else → (middle rows)
{
if (col==0 || col==4)
cout << *
else
cout << " ";
r0
r1
r2
r3

```
for ( int r = 0; r < 4; r++ )
{
    for ( int c=0; c<5; c++ )
    {
        if (r==0 || r==3)
            cout << "*";
        else
        {
            if (c==0 || c==width-1)
                cout << "*";
            else
                cout << " ";
        }
    }
    cout << endl;
}
```

colums = 5

width

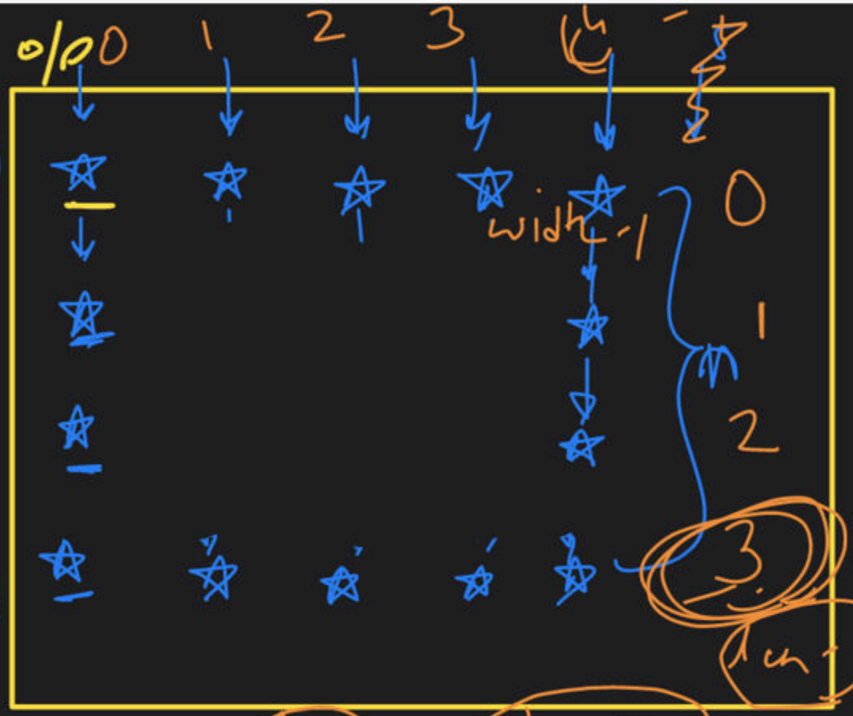
r = 0

r = 1

r = 2

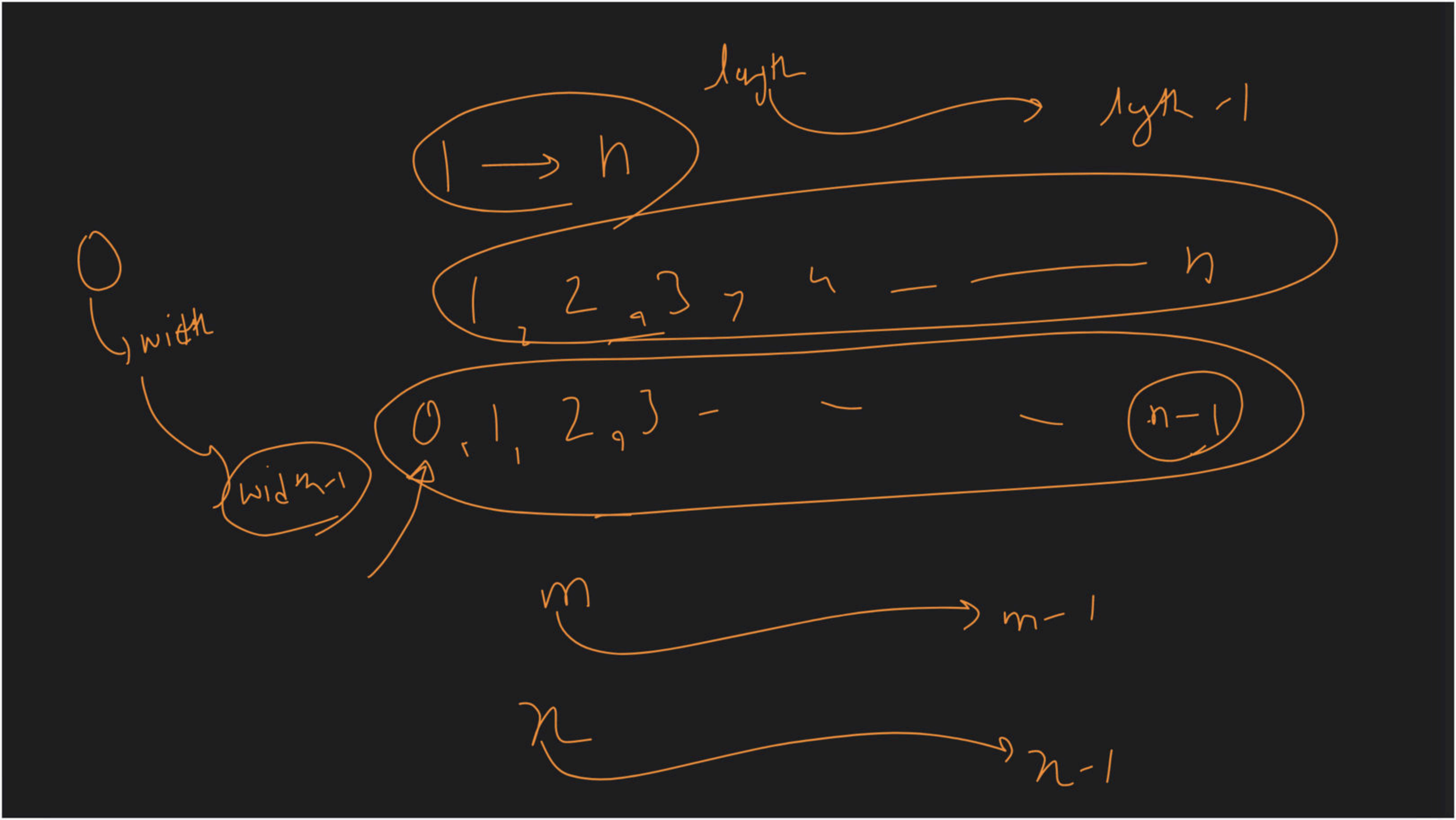
r = 3

width-1



r == 0

first


layer
layer -1
1 → h
0
width
1 2 3 4 — — — h
0 1 2 3 — — — n-1
width-1
m
m-1
n
n-1


1, 2, 3, 4, — — — n
(-1)
0, 1, 2, 3 — — — n-1

Hollow Square

→ total rows → n = 5

→

r0 → 5 ☆

r1 → 1 ☆ 3 sp 1 ☆

r2 → 1 ☆ 3 sp 1 ☆

r3 → 1 ☆ 3 sp 1 ☆

r4 → 5 ☆

Ohh! -

↳ First / Last → col → ☆

↳ middle → rows

first / Last

col

☆

otherwise

Space

1 min

c0 c1 c2 c3 c4

r0 → ☆ ☆ ☆ ☆ ☆

r1 → ☆ — — — ☆

r2 → ☆ — — — ☆

r3 → ☆ — — — ☆

r4 → ☆ ☆ ☆ ☆ ☆

for (r = 0 → r < 5)

{

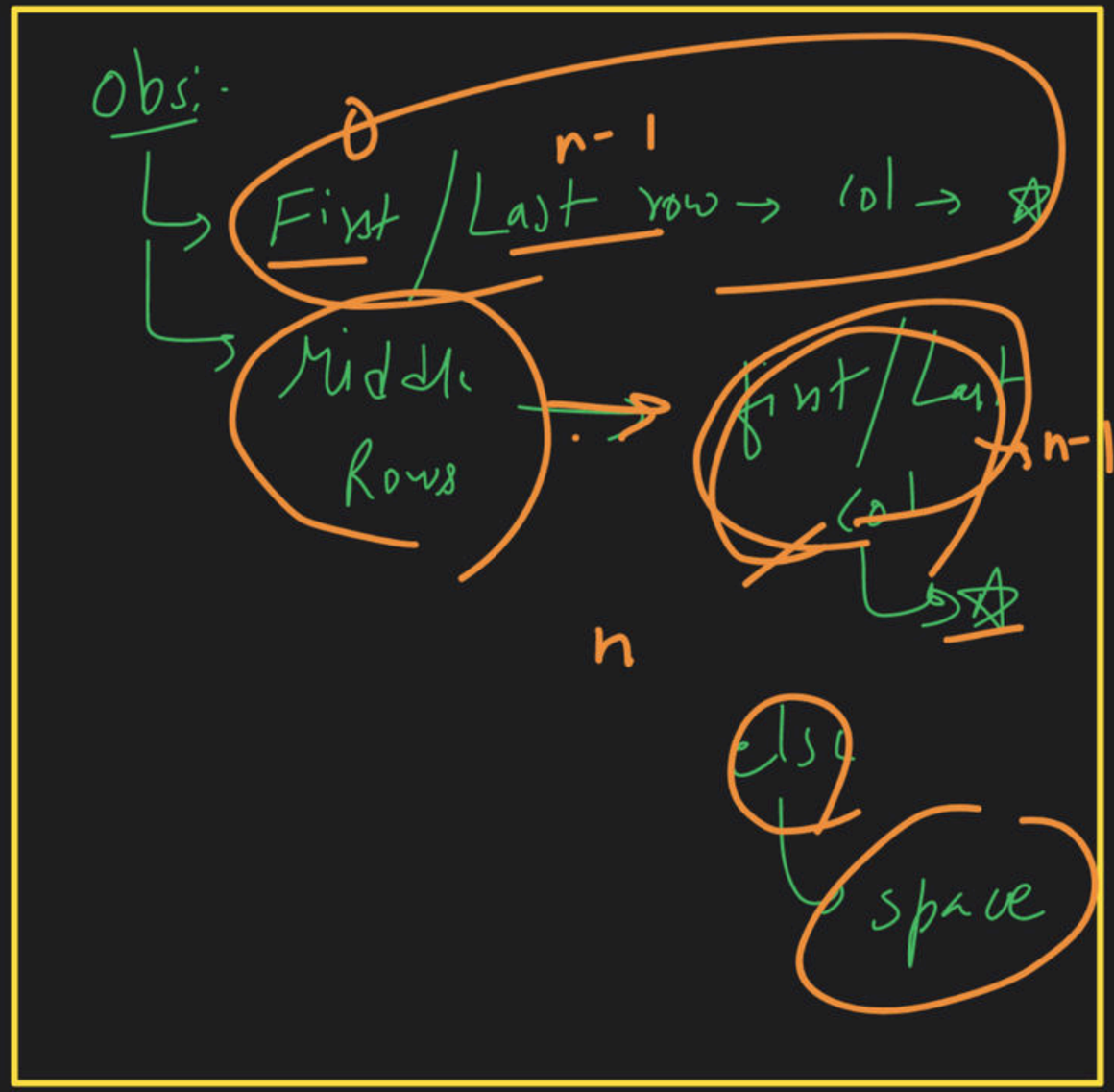
for (c = 0 → c < 5)

{

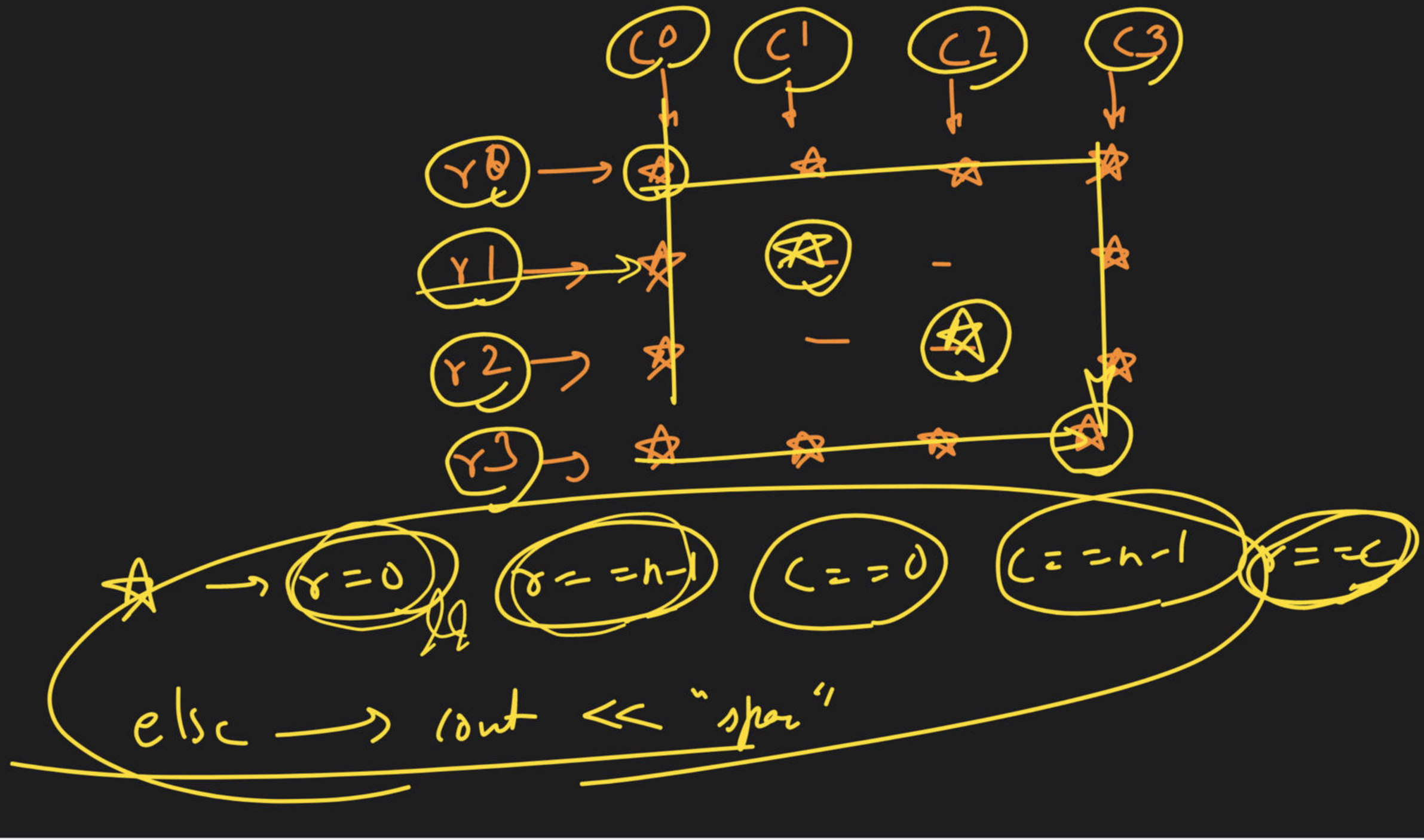
// Logic

}

}




```
n
if ( r==0 || r==n-1)
    cout << ☆
else
{
    if (c==0 || c==n-1)
        cout << ☆
    else
        cout << " "
}
```


c0
c1
c2
c3
r0
r1
r2
r3
r==0
r==n-1
c==0
c==n-1
r==c
else → cout << "spar"

Half Pyramid

1 min Break



inner → simple → diagg → no of col vary karte hai

① total rows → n = 5

② r0 → 1 ☆
r1 → 2 ☆
r2 +1 → 3 ☆
r3 +1 → 4 ☆
r4 +1 → 5 ☆

row → row+1 ☆
stars

c0 c1 c2 c3 c4
r0 → ☆
r1 → ☆ ☆
r2 → ☆ ☆ ☆
r3 → ☆ ☆ ☆ ☆
r4 → ☆ ☆ ☆ ☆ ☆

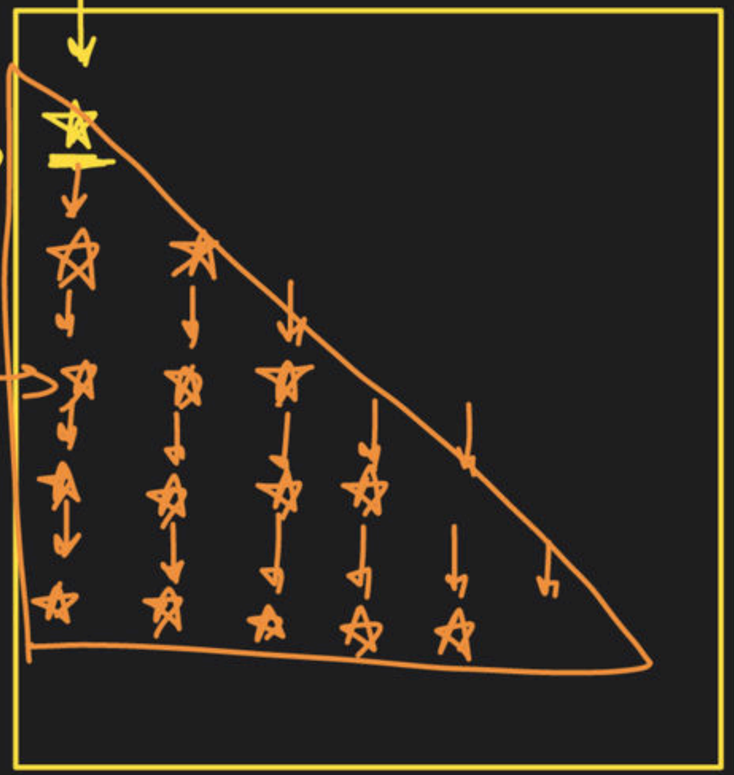
```
for (r=0 ⟶ r<n)
{
    for (c=0 ⟶ c<row+1)
    {
        cout << ☆
    }
}
```

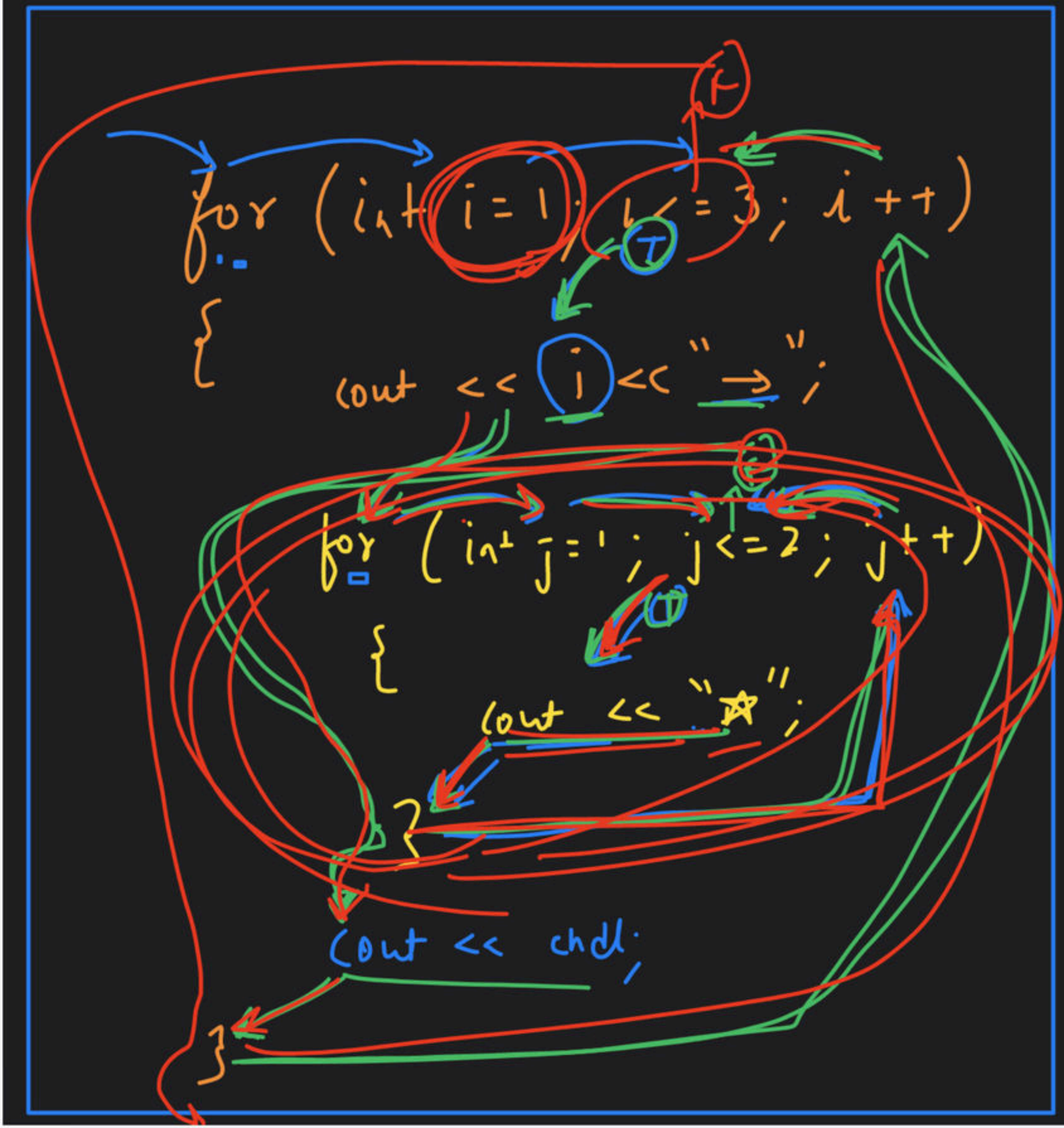
n=5

```
for ( r=0 ; r<n ; r++ )
{
    for ( c=0 ; c<r+1 ; c++ )
    {
        cout << "*" ;
    }
    cout << endl;
}
```

o/p

r=0
r=1
r=2
r=3
r=4
r=5





i=3
3<=3 → T
j=1
1<=2 → T
j=2
2<=2 → T
j=3
3<=2 → F
i=4
4<=3
→ F

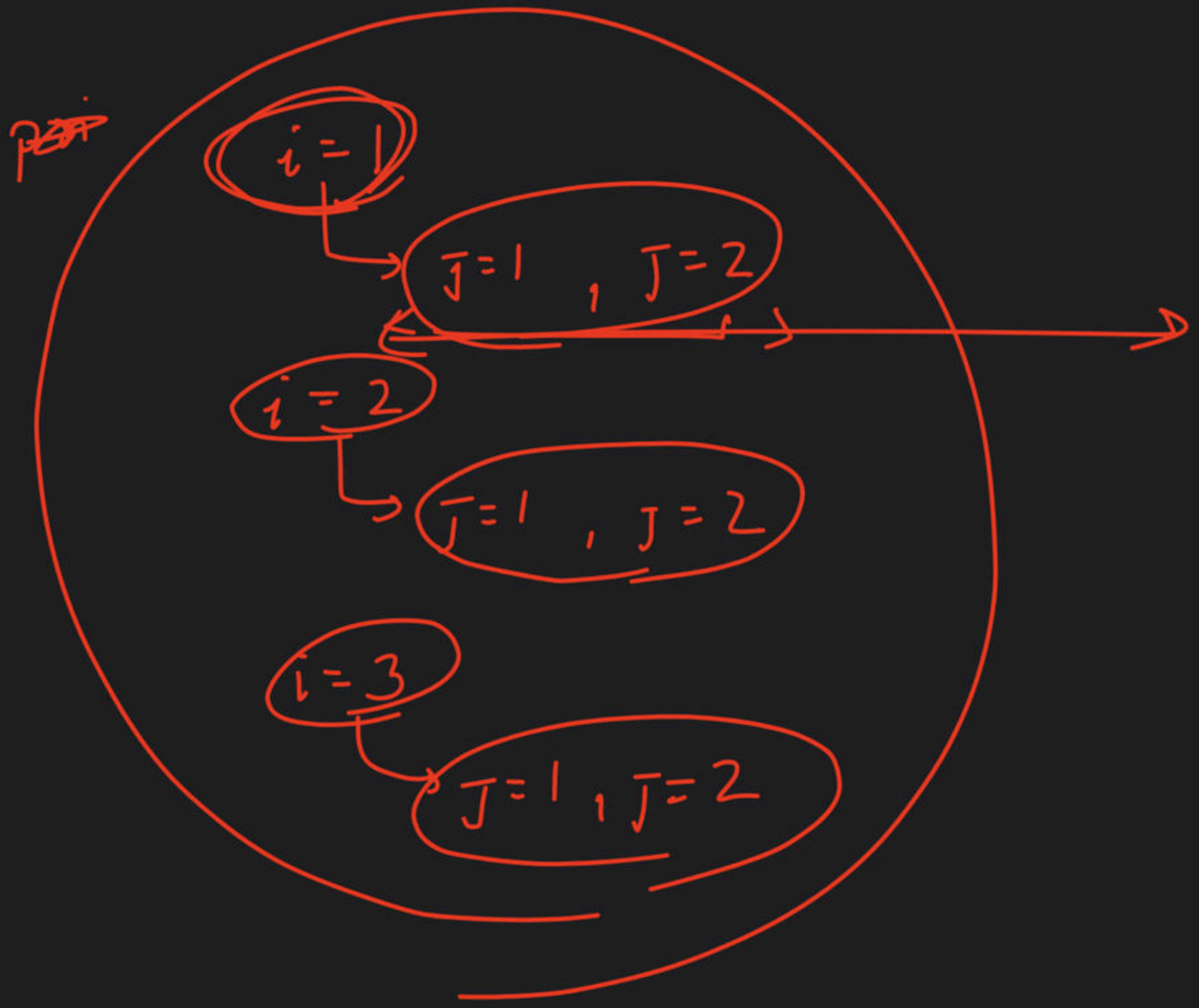
i=2
2<=3
→ T
j=1
1<=2
→ T
j=2
2<=2
→ T
j=3
3<=2
→ F

i=1
1<=3 → T
j=1
1<=2
→ T
j=2
2<=2
→ T
j=3
3<=2
→ F

output:

1 → ☆☆
2 → ☆☆
3 → ☆☆

1 → ☆☆
2 → ☆☆
3 → ☆☆


i = 1
j = 1 , j = 2
i = 2
j = 1 , j = 2
i = 3
j = 1 , j = 2

row = 0

Rule

→ total rows ⇒ n = 5

formula

Ohs

+1

row=0 r0

row=1 r1

row=2 r3

row=3+1 r3

row=4+1 r4

1 ⭐

2 ⭐

3 ⭐

4 ⭐

5 ⭐

1 star

each row

star

row+1

org re?

r0 → ⭐

r1 → ⭐ ⭐

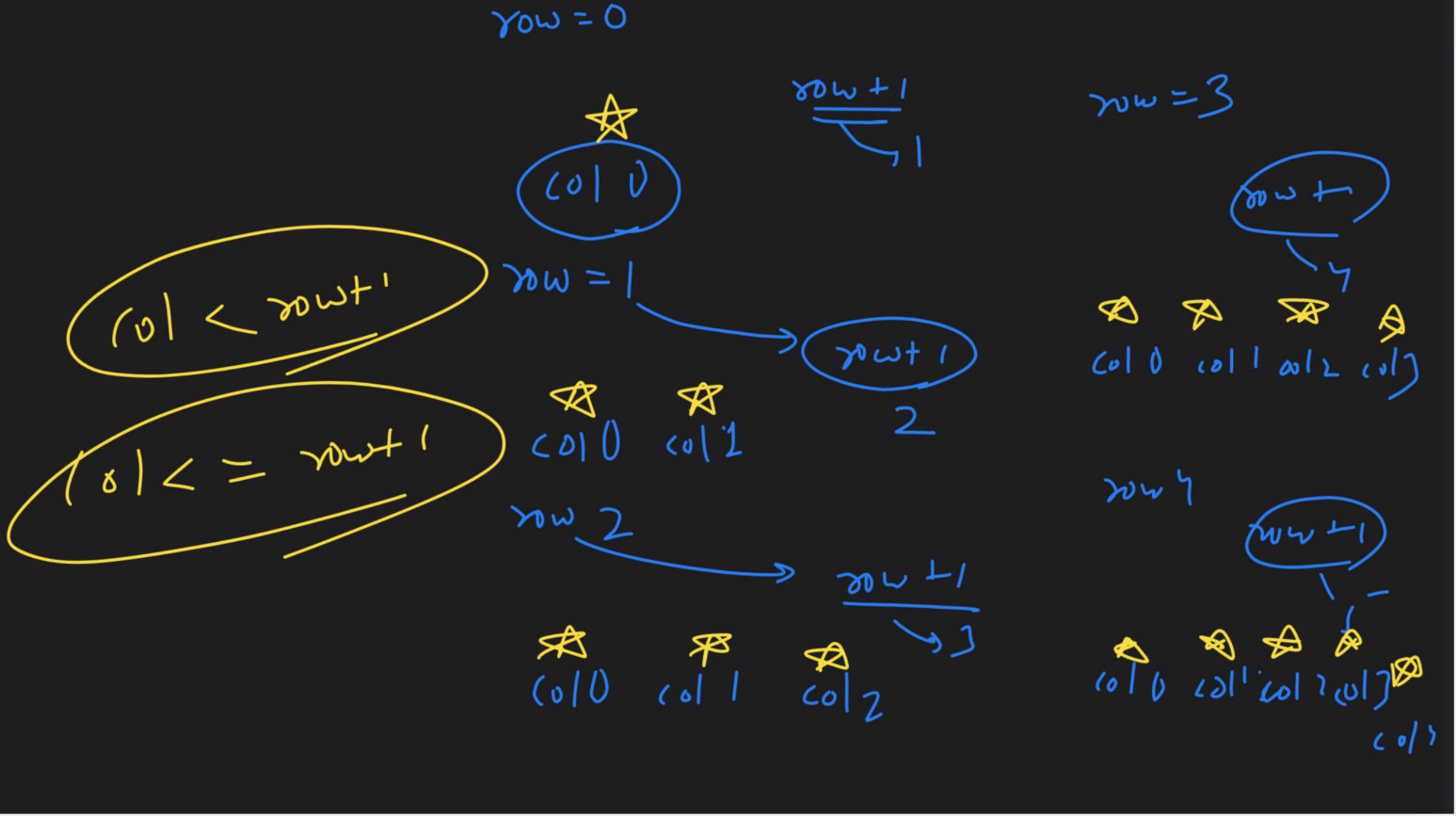
r2 → ⭐ ⭐ ⭐

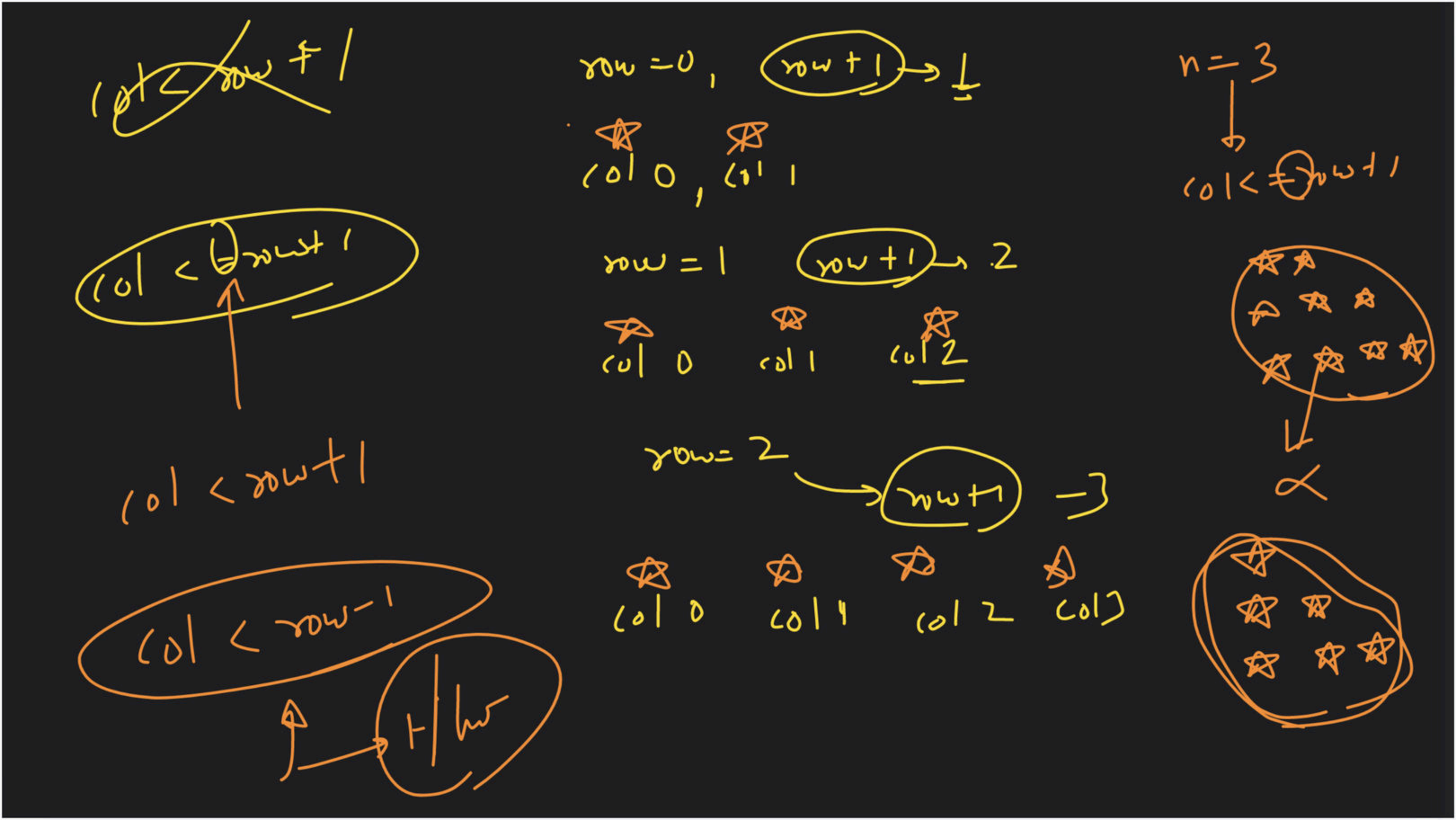
r3 → ⭐ ⭐ ⭐ ⭐

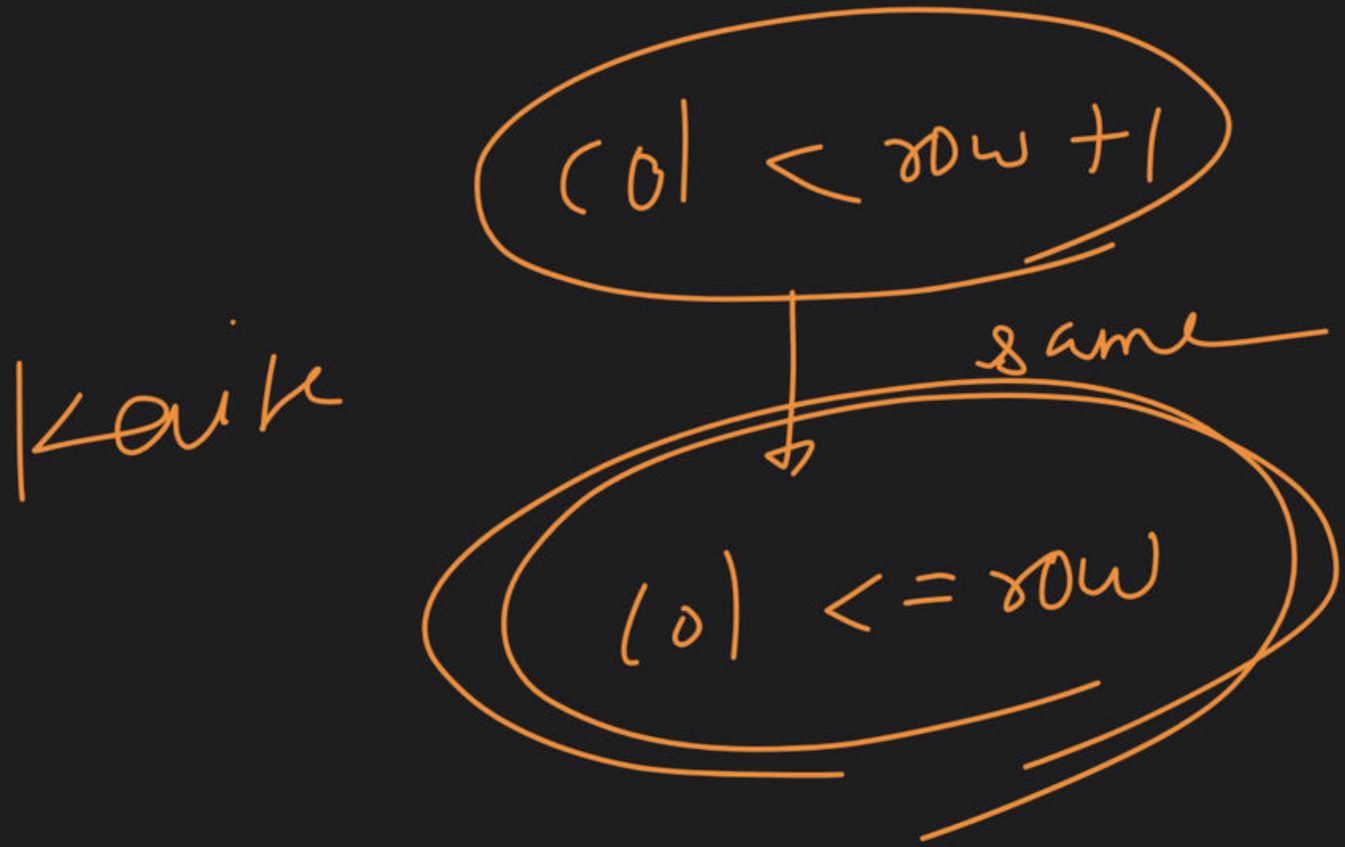
r4 → ⭐ ⭐ ⭐ ⭐ ⭐

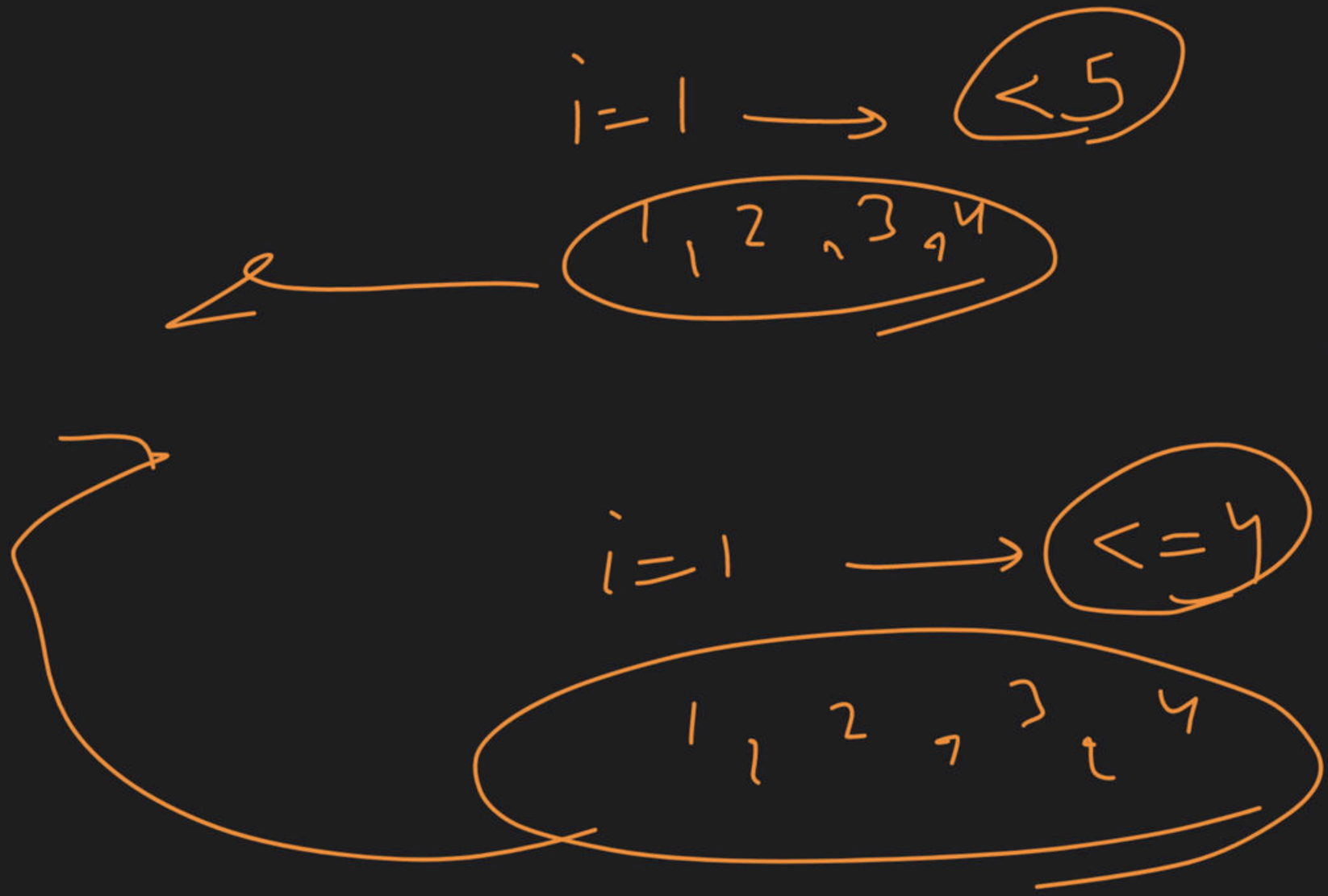
```
for ( r = 0 ⟶ r < n )
{
    for ( col = 0 ; col < row+1 ; col++ )
    {
        cout << ⭐ ;
    }
}
```

formula


row = 0
row + 1
1
col 0
row = 1
col < row + 1
col <= row + 1
col 0
col 1
row + 1
2
row = 2
row + 1
col 0
col 1
col 2
row = 3
row + 1
4
col 0
col 1
col 2
col 3
row 4
row + 1
col 0
col 1
col 2
col 3
col 4


col < row + 1
row = 0, row + 1 → 1
col 0, col 1
n = 3
col <= row + 1
col < row + 1
row = 1 row + 1 → 2
col 0 col 1 col 2
col < row + 1
row = 2 row + 1 →
col 0 col 1 col 2 col 3
col < row - 1
+ / -


col < row + 1
same
col <= row


i=1
<5
1 2 3 4
south
i=1
<=4
1 2 3 4

→ inverted half Pyramid

Socho

① total rows → n=4

n=4
r=0
☆ = 4 → target
n-r → 4

formula

Obs

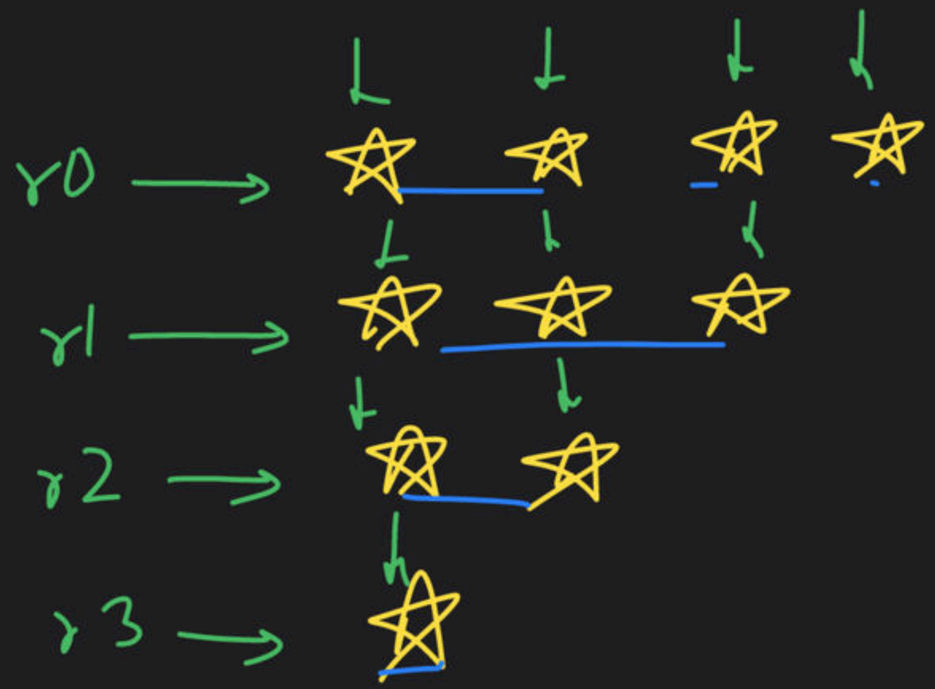
n - row

r0 → 4 ☆
r1 → 3 ☆
r2 → 2 ☆
r3 → 1 ☆

n=4
r=1
n-r=4-1=3

n=4 r=2
4-2

r=3
n=1
4-3




```
for ( r=0 ; r<n; r++)
{
    for ( c=0; c<n-row; c++)
    cout ☆;
}
```

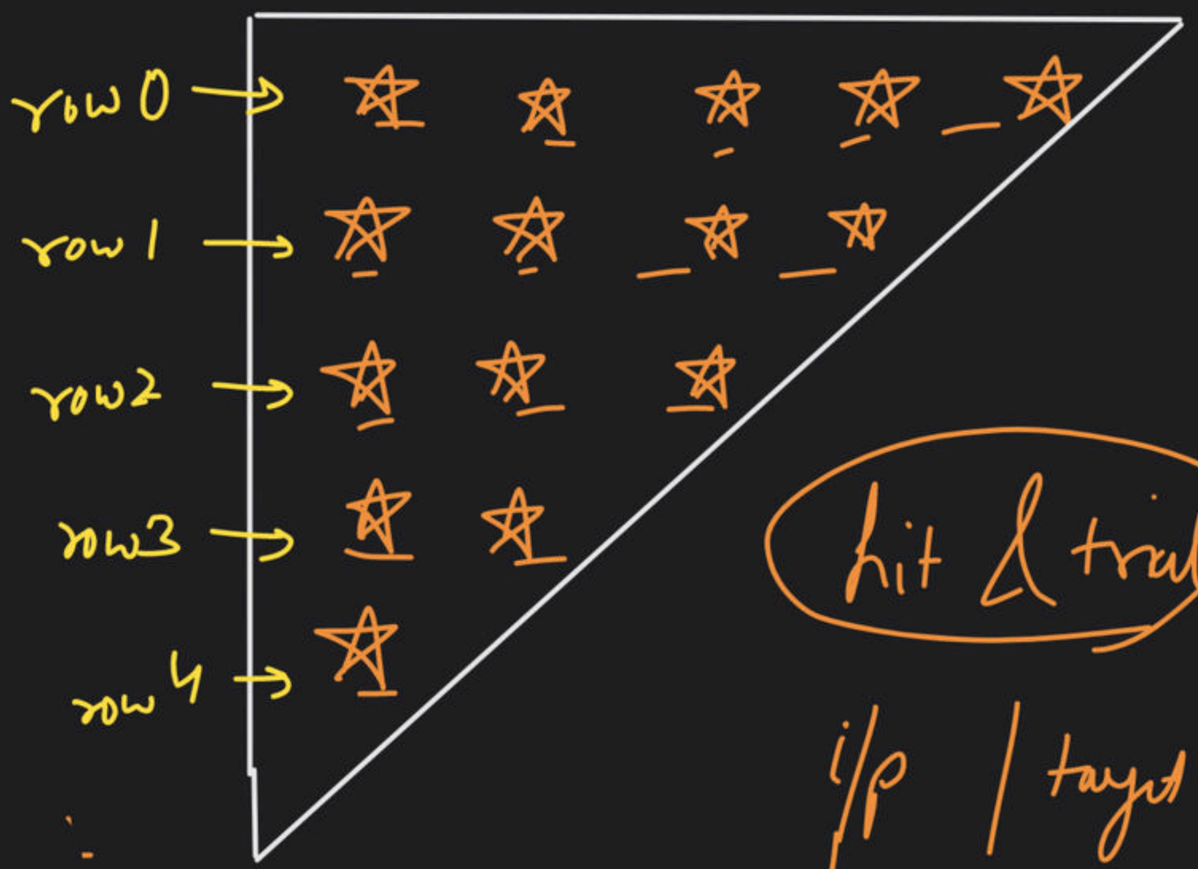
→ total rows => n = 5

Formula

Obs:-

i/p
n=5
r=4
5-4 → 1

target
1



hit & trial

row 0 → 5
row 1 → 4
row 2 → 3
row 3 → 2
row 4 → 1

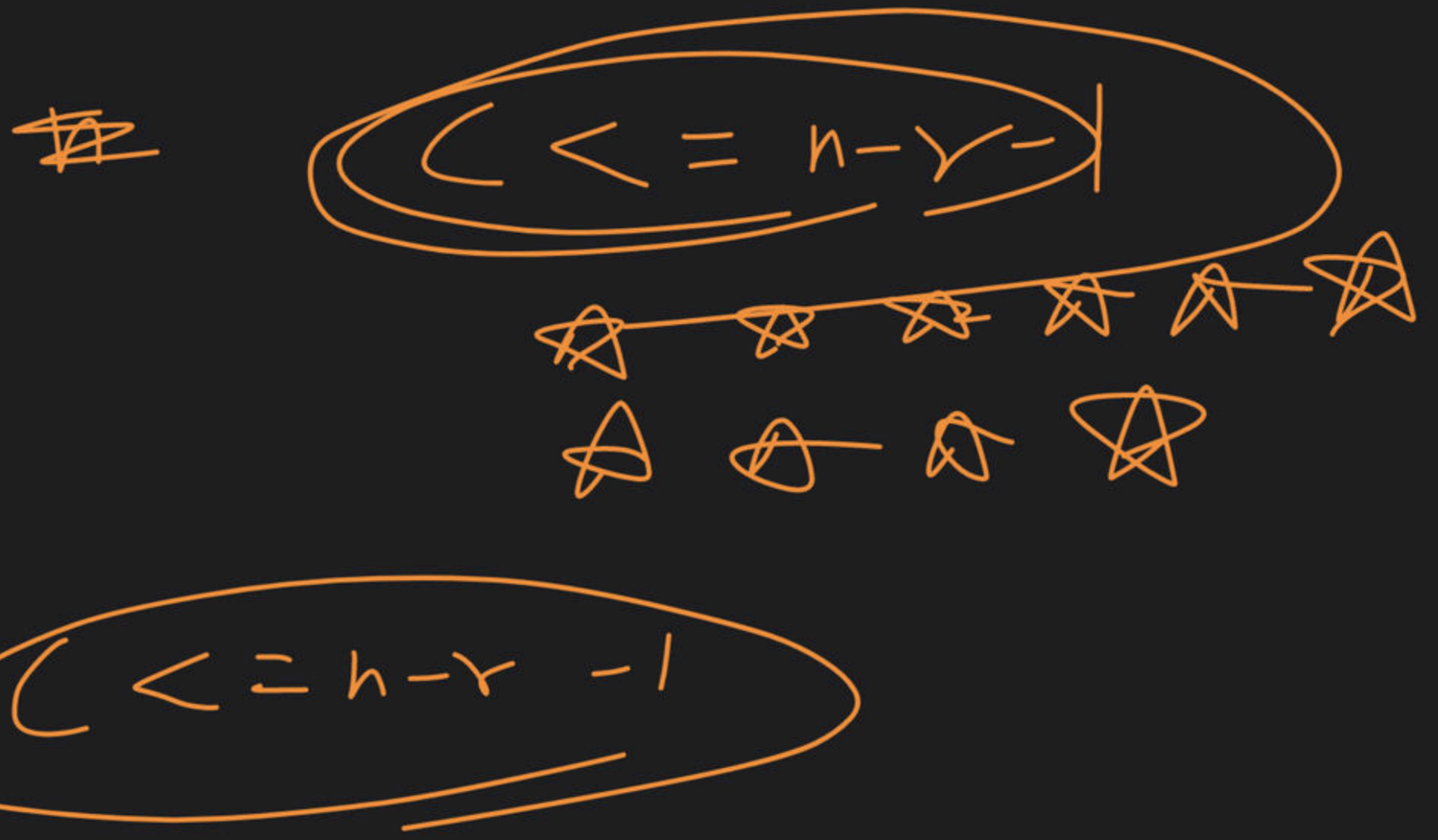
i/p
n=5
row=3
n-r
n-row → 5-3 = 2

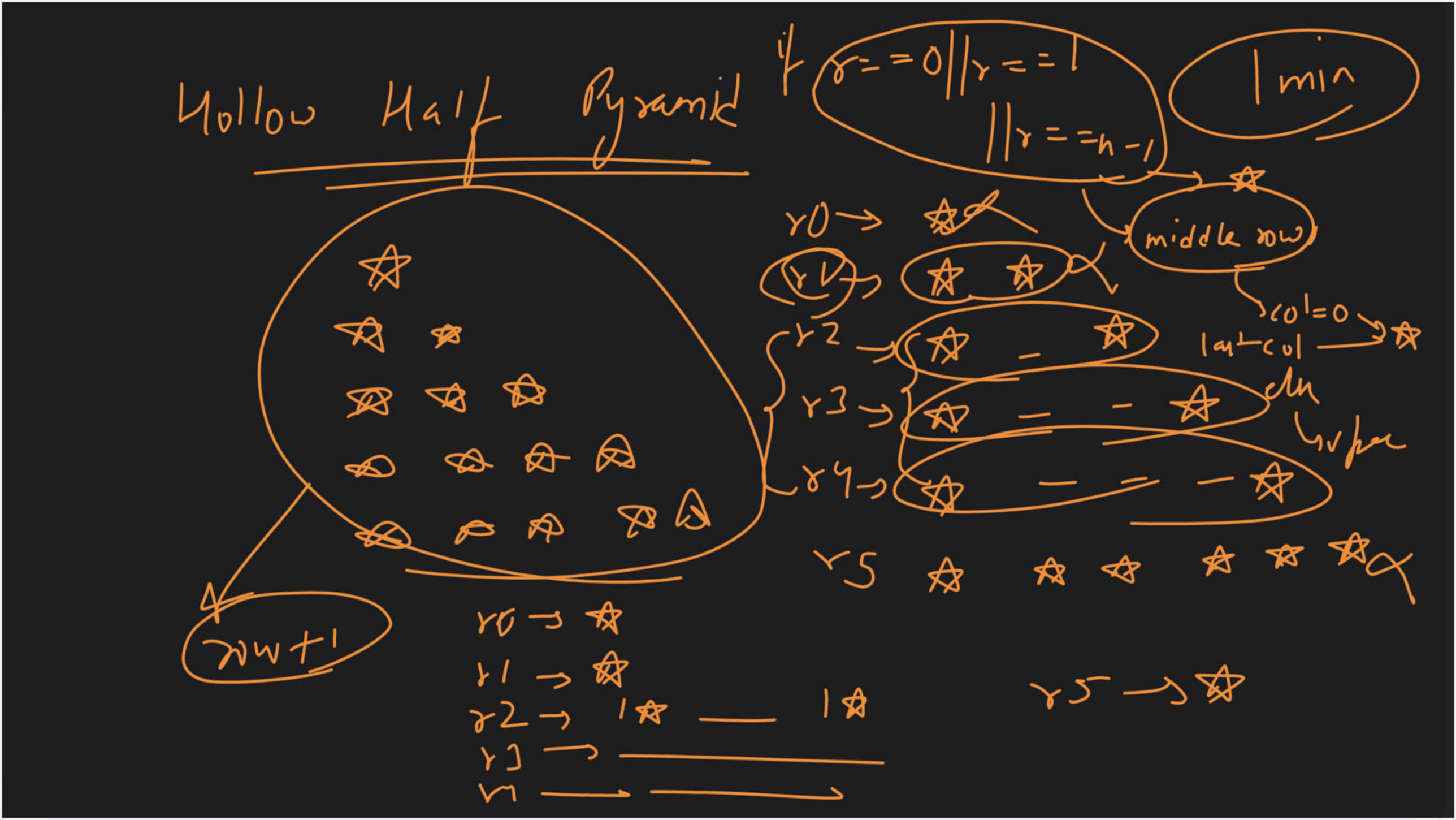
target
2

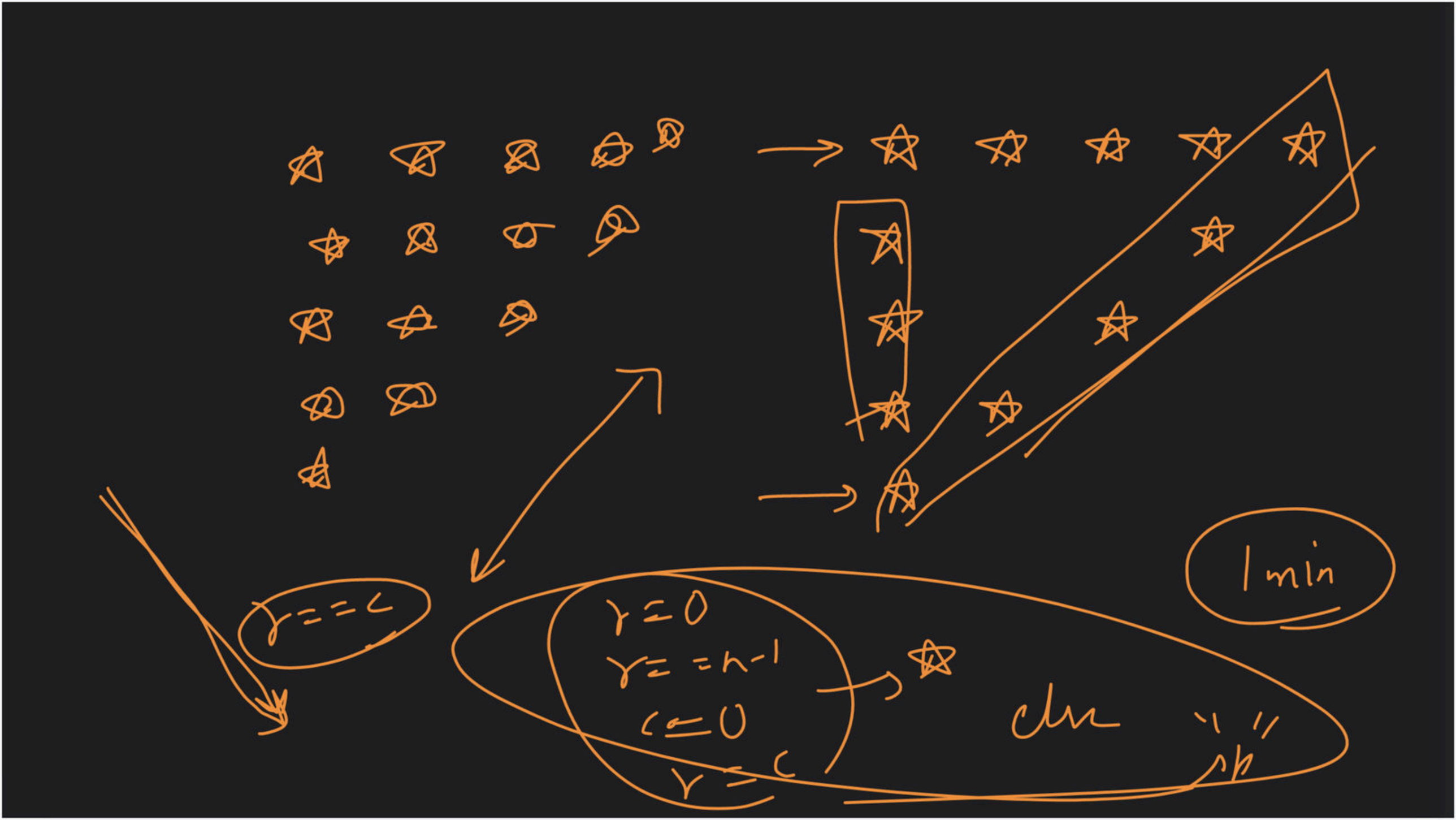
i/p | target

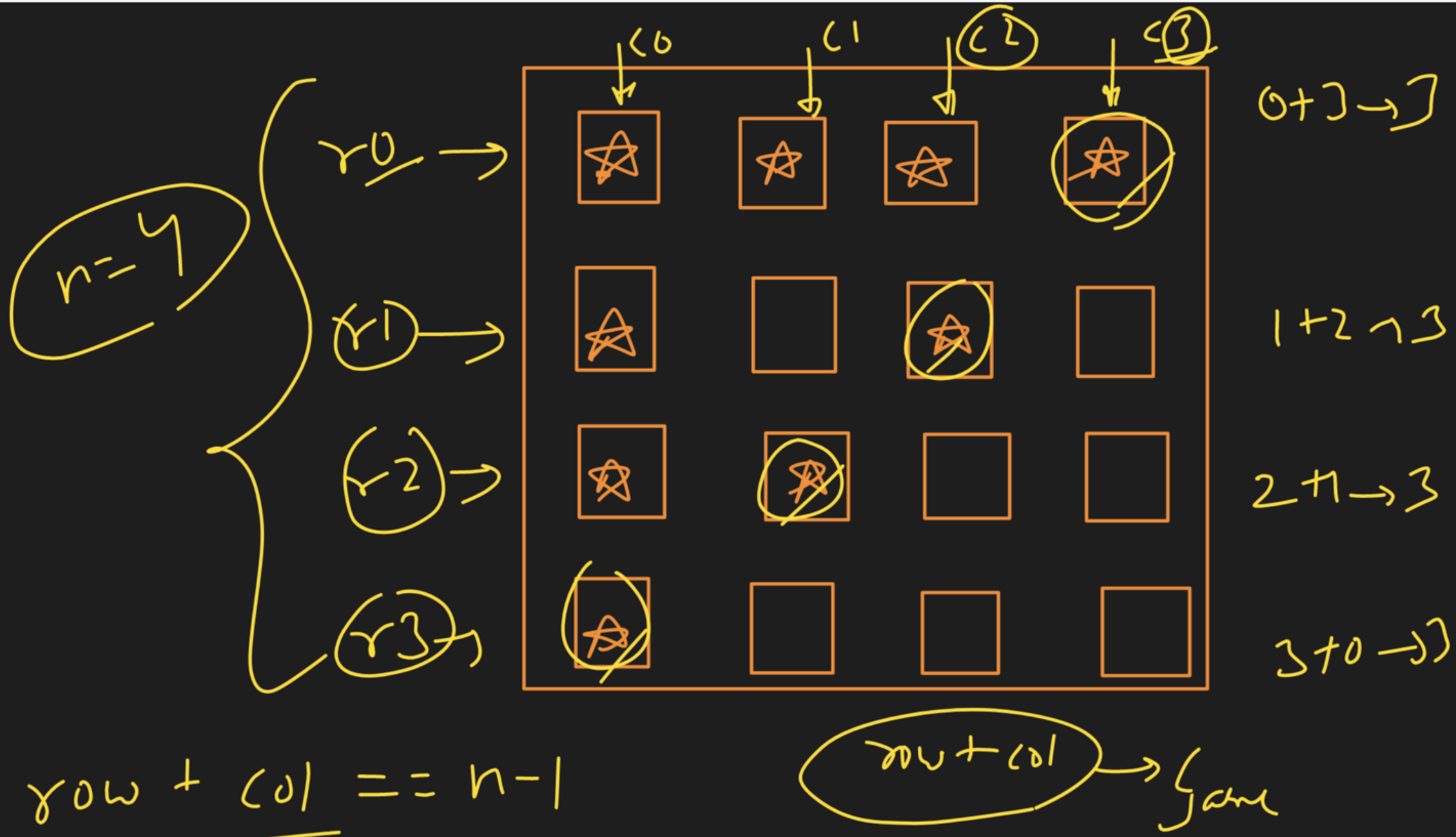
n=5
r=0
n-r → 5
n-r → 5

5

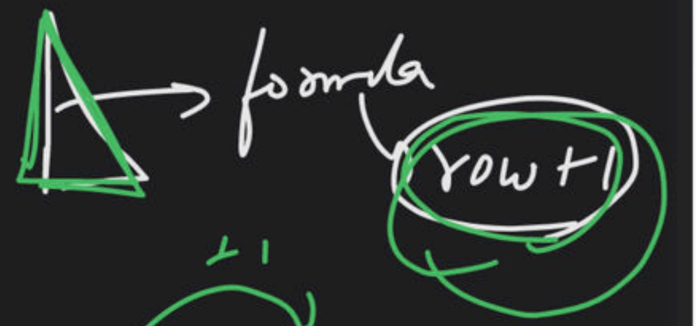

(<= n-2-1
(<= h-2 -1


Hollow Half Pyramid
if r==0||r==1
||r==n-1
print
middle row
r0→
r1→
r2→
r3→
r4→
r5
col=0
last col
row+1

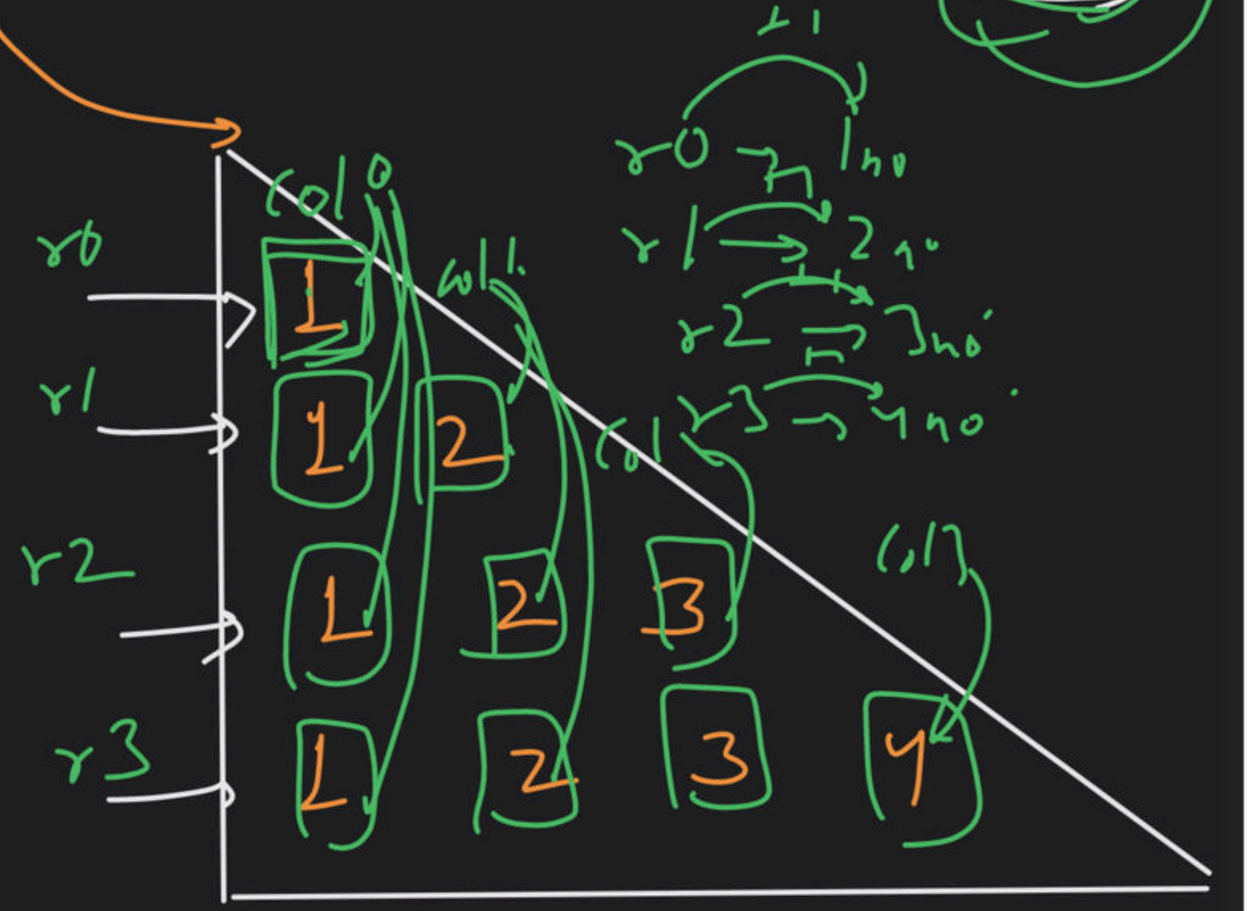

r==c
r=0
r==n-1
c=0
r==c
1 min

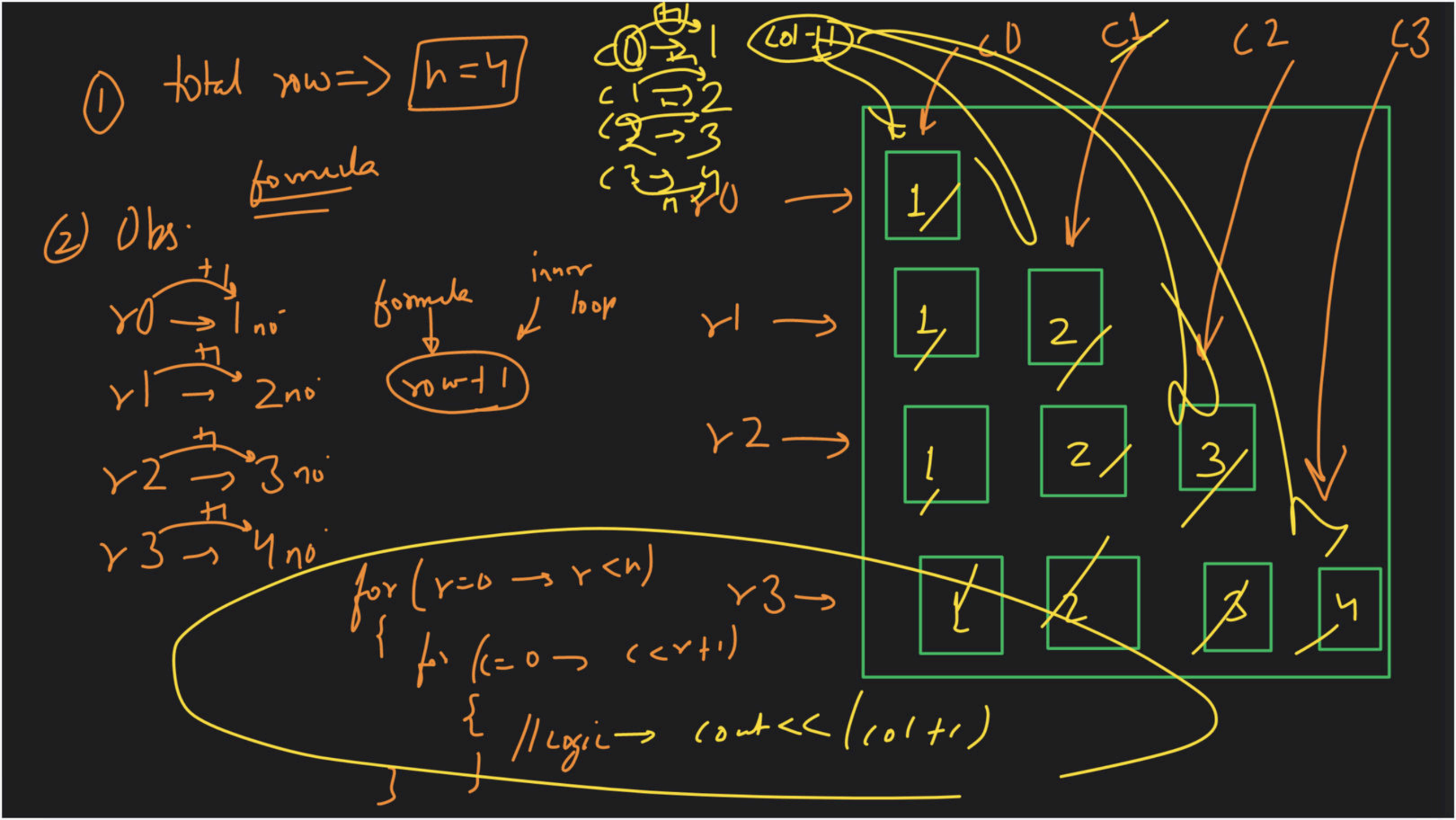

c0
c1
c2
c3
r0
r1
r2
r3
n=4
0+3→3
1+2→3
2+1→3
3+0→3
row + col == n-1
row+col → same

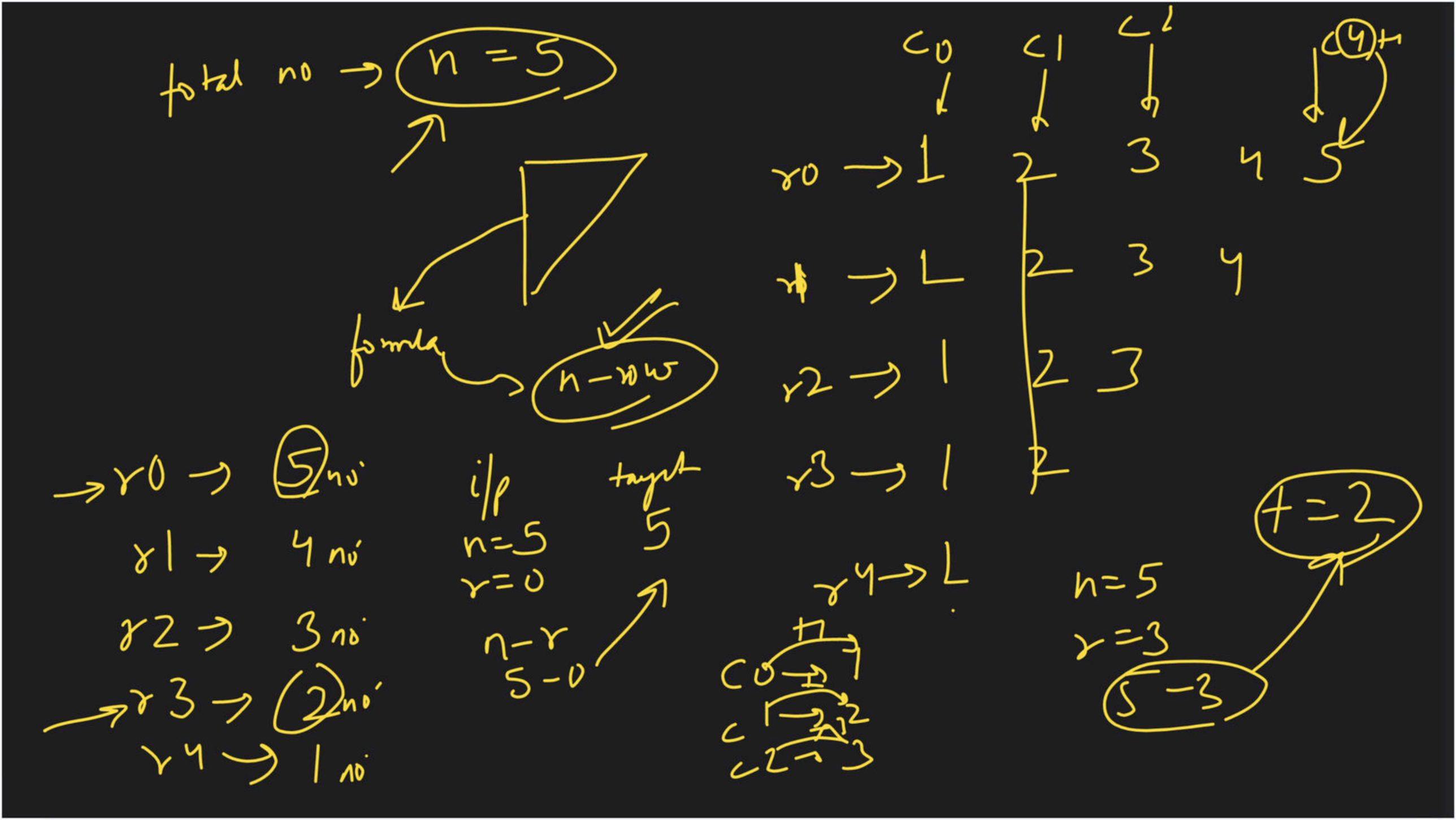
→ Numeric Half Pyramid




```
h=4
for(r=0 → r<4)
{
    for(col 0 → col<row+1)
    {
        cout << (col+1)
    }
}
```




① total row => n = 4
formula
② Obs.
r0 → 1 no.
r1 → 2 no.
r2 → 3 no.
r3 → 4 no.
+1
formula
inner loop
row+1
col=0
col=1
col=2
col=3
r0 →
r1 →
r2 →
r3 →
1
1 2
1 2 3
1 2 3 4
for(r=0 → r<n)
{
for(c=0 → c<r+1)
{ //Logic → cout<<(col+1)
}
}


total no → n = 5
formula
n-1 to 0
c0
c1
c2
c3
c4
r0 → 1 2 3 4 5
r1 → 1 2 3 4
r2 → 1 2 3
r3 → 1 2
r4 → 1
→ r0 → 5 no
r1 → 4 no
r2 → 3 no
→ r3 → 2 no
r4 → 1 no
i/p
n=5
r=0
target
5
n-r
5-0
r4 → 1
c0 → 1
c1 → 2 1
c2 → 3 2 1
n=5
r=3
5-3
i=2

for (r=0 ⟶ r<n)

{

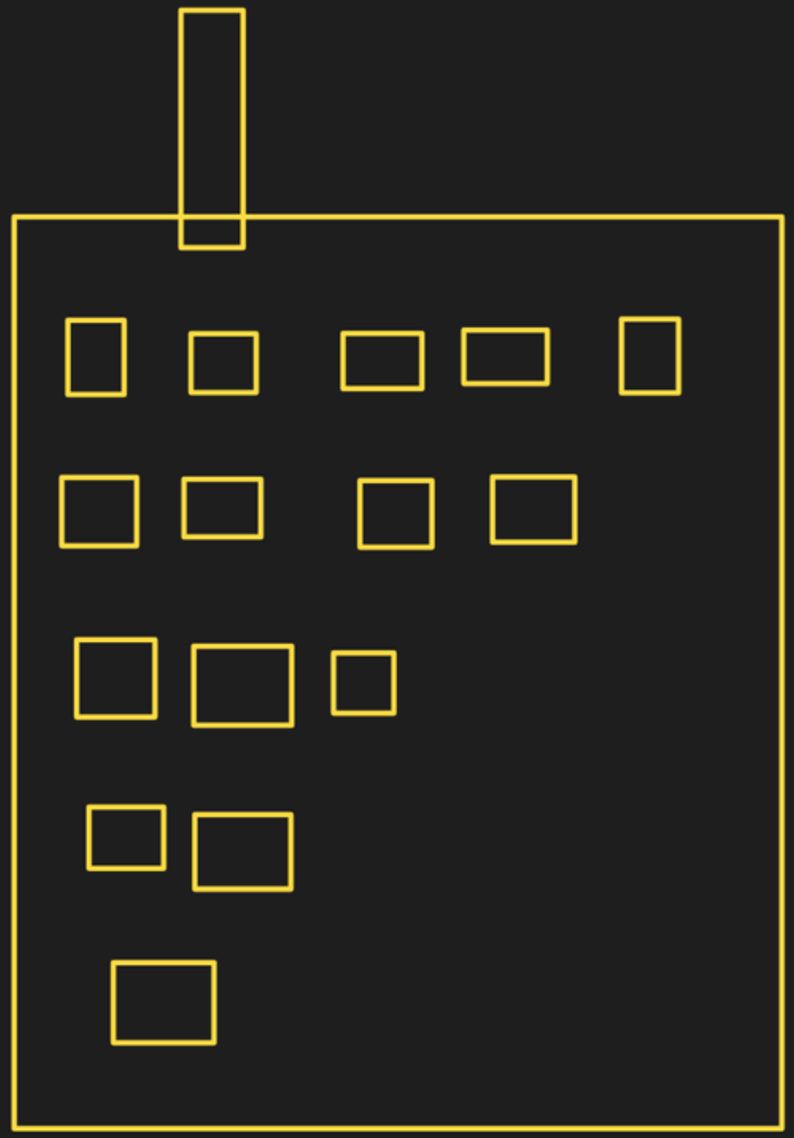
for (c=0 ⟶ c<n-no)

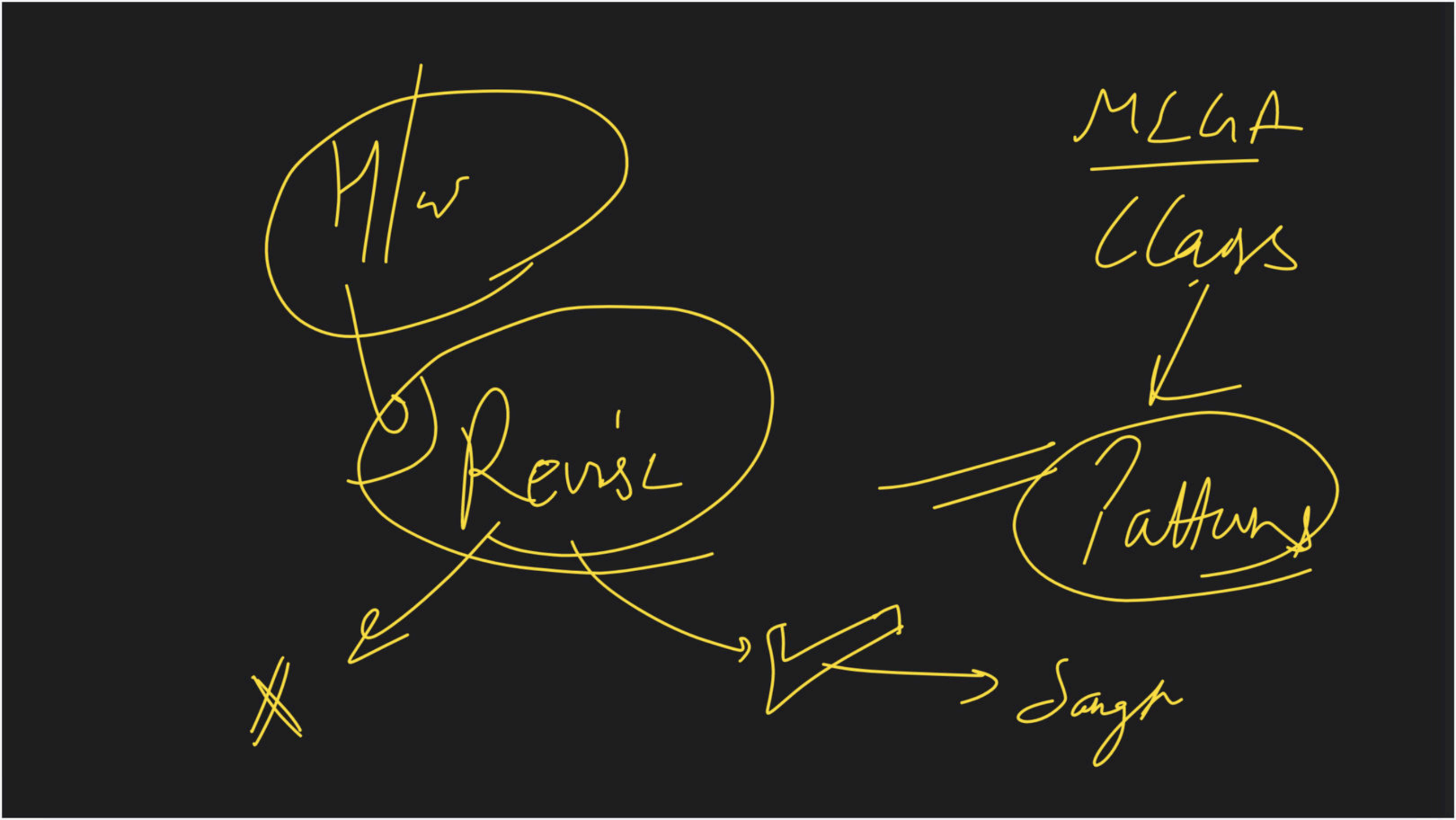
{

cout ⟶ *(col+r)

}

}




MVC
Revise
MCA
Class
Pattern
Sangh